spark on yarn模式内存细分

已于 2022-03-23 17:57:44 修改
编辑

flink 同时被 2 个专栏收录 ¥9.90 ~~¥99.00~~ 59 篇文章

摘要 本文详细介绍了Spark on YARN模式下的内存配置，包括Executor的Storage Memory和Shuffle Memory。讲解了Storage Memory用于缓存RDD和DataFrame，其大小受`spark.storage.memoryFraction`和`spark.storage.safetyFraction`影响。Shuffle Memory用于shuffle过程，其默认大小为Executor内存的一部分。...

Spark 基本概念

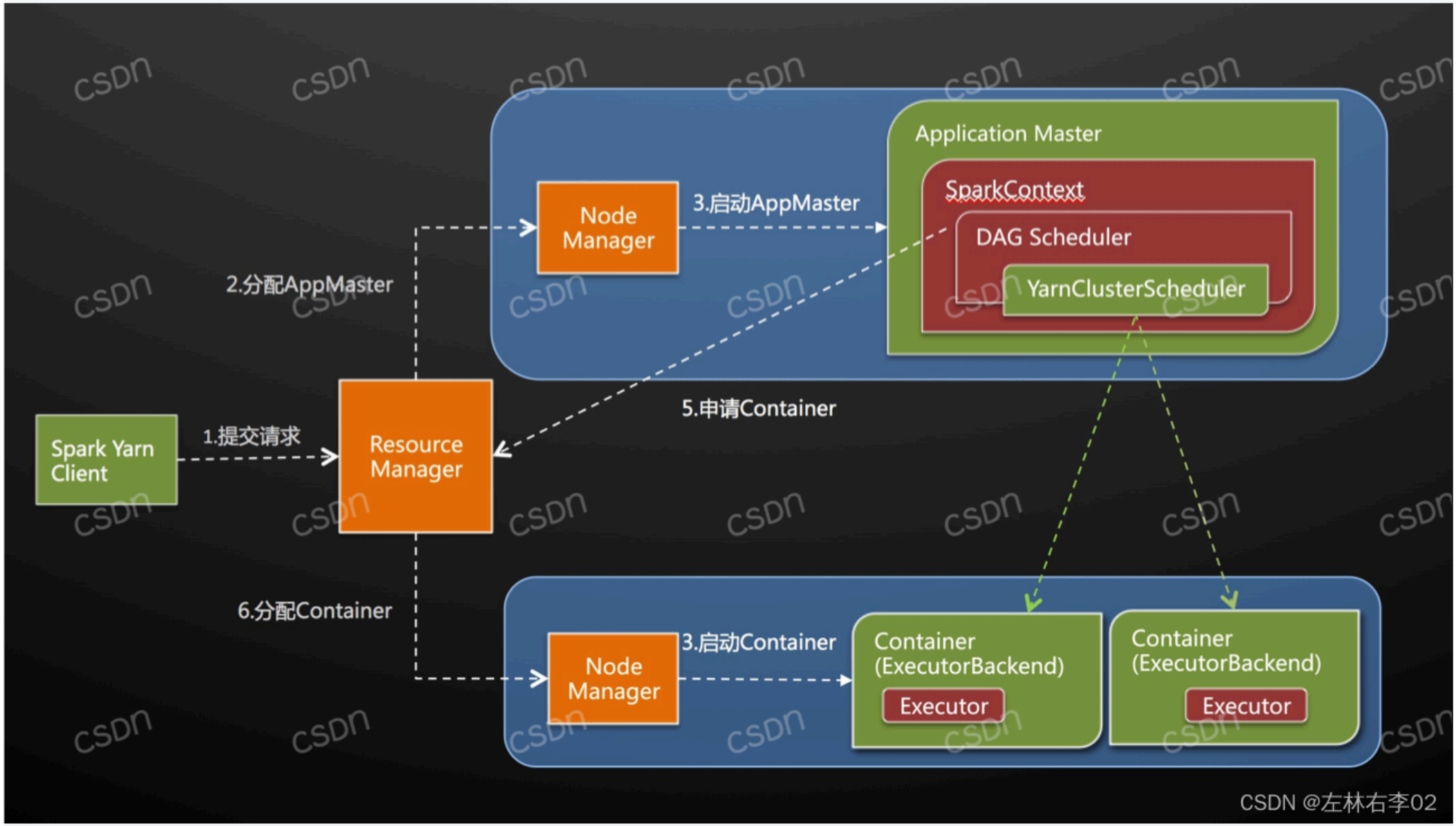
在讲解Spark作业memory使用前，确保理解以下Spark 基本概念：

- Application： 基于Spark的用户程序，包含了一个driver program 和 集群中多个的executor
- Driver：运行Application的main()函数并且创建SparkContext，通常用SparkContext代表Driver Program
- Executor： 是为某Application运行在worker node上的一个进程，该进程负责运行Task，并且负责将数据存在内存或者磁盘上。每个Application都有各自独立的executors。
- Task： 被送到某个executor上的工作单元
- RDD：Spark的基本计算单元，可以通过一系列算子进行操作

Spark on Yarn 运行模式及线上配置

内容来源：csdn.net
作者昵称：左林右李02
原文链接：https://blog.csdn.net/u011624157/article/details/123690957
作者主页：https://blog.csdn.net/u011624157

Spark on YARN的运行架构图：



Spark on YARN模式下，每一个Spark Executor将作为一个YARN Container 运行。

那么，每一个Executor的资源分配将受限于Container可使用的资源。

yarn内存配置

内容来源：csdn.net
作者昵称：左林右李02
原文链接：https://blog.csdn.net/u011624157/article/details/123690957
作者主页：https://blog.csdn.net/u011624157

```
yarn.nodemanager.resource.memory-mb: NM能够分配给所有该节点上Container的物理内存

yarn.nodemanager.resource.cpu-vcores: NM能够分配给所有该节点上Container的CPU 核数

yarn.scheduler.minimum-allocation-mb: RM分配给container所申请的内存保证是该值的倍数，设置值为256MB

yarn.scheduler.increment-allocation-mb: container能够向RM申请的内存的增量值，设置为256MB

yarn.scheduler.maximum-allocation-mb: 调度时一个container能够向RM申请的最大内存，设置为16384M = 16G: 即每一个Spark Executor、Driver最大使用内存为

yarn.scheduler.maximum-allocation-vcores: 调度时一个container能够向RM申请的core个数，设置为8 ：即每一个Spark Executor最大使用core数为8
```

AI写代码java

YARN为Executor分配资源配置

```java
// In YarnSparkHadoopUtil.scala
val MEMORY_OVERHEAD_FACTOR = 0.10
val MEMORY_OVERHEAD_MIN = 384

// In YarnAllocator.scala
// Executor memory in MB.
protected val executorMemory = args.executorMemory
// Additional memory overhead.
protected val memoryOverhead: Int = sparkConf.getInt("spark.yarn.executor.memoryOverhead",
  math.max((MEMORY_OVERHEAD_FACTOR * executorMemory).toInt, MEMORY_OVERHEAD_MIN))
// Number of cores per executor.
protected val executorCores = args.executorCores
// Resource capability requested for each executors
private val resource = Resource.newInstance(executorMemory + memoryOverhead, executorCores)
```

AI写代码java

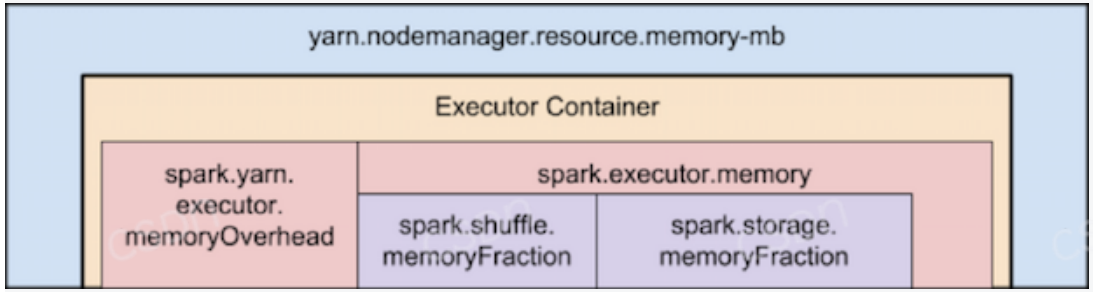
内容来源：csdn.net
作者昵称：左林右李02
原文链接：https://blog.csdn.net/u011624157/article/details/123690957
作者主页：https://blog.csdn.net/u011624157

可见：

YARN真正分配给Executor的Memory为：

`executor申请的memory，与executor堆外内存的总和， 而默认堆外内存executor.memoryOverhead = max { 1/10 * executor.memory, 384M }`

spark on yarn内存分布(executor进一步细分)



`executor可用内存 = spark.executor.memoryOverhead + spark.executor.memory`

`spark.executor.memory = spark.shuffle.memoryFraction + spark.storage.memoryFraction + other (JVM 堆栈空间)`

Storage Memory：

用于缓存需要复用的RDD 、DataFrame

那么storage memory使用的空间为多少呢：

```
/** Return the total amount of storage memory available. */
private def getMaxMemory(conf: SparkConf): Long = {
  val memoryFraction = conf.getDouble("spark.storage.memoryFraction", 0.6)
  val safetyFraction = conf.getDouble("spark.storage.safetyFraction", 0.9)
  (Runtime.getRuntime.maxMemory * memoryFraction * safetyFraction).toLong
}
```

AI写代码java

可见，默认情况下

真正用于storage的内存为：

`实际分配得到的内存 * spark.storage.memoryFraction(默认0.6) * spark.storage.safetyFraction(默认0.9)`

需要注意：

内容来源：csdn.net
作者昵称：左林右李02
原文链接：https://blog.csdn.net/u011624157/article/details/123690957
作者主页：https://blog.csdn.net/u011624157

用于缓冲数据的内存空间不应该超过JVM老年代的大小

ResourceManager、NodeManager使用Parallel Scavenge + Parallel Old 组合进行GC管理，新生代与老年代比例取默认值，为1：2。

即，Executor（Container）内能够实际用于缓存数据的最大值为 executor.memory * (2/3)

总结：用于缓存rdd的内存最大为：

> min(executor_memory*spark.storage.memoryFraction * spark.storage.safetyFraction , JVM老年代大小)

如果ExecutorMemory配置为8G，那么内存中能缓存的rdd最大为min(8G*0.60.9 , 8G*2/3)=min(4.32G , 5.33G)=4.32G

Shuffle Memory：

用于shuffle时，缓存从map端拉取过来的数据

那么shuffle memory使用的空间为多少呢：

ShuffleMemoryManager

```
/**
 * Figure out the shuffle memory limit from a SparkConf. We currently have both a fraction
 * of the memory pool and a safety factor since collections can sometimes grow bigger than
 * the size we target before we estimate their sizes again.
 */
def getMaxMemory(conf: SparkConf): Long = {
  val memoryFraction = conf.getDouble("spark.shuffle.memoryFraction", 0.2)
  val safetyFraction = conf.getDouble("spark.shuffle.safetyFraction", 0.8)
  (Runtime.getRuntime.maxMemory * memoryFraction * safetyFraction).toLong
}
```

AI写代码java

内容来源：csdn.net
作者昵称：左林右李02
原文链接：https://blog.csdn.net/u011624157/article/details/123690957
作者主页：https://blog.csdn.net/u011624157

可见：默认情况下，真正用于shuffle的内存为：

```
实际分配得到的内存 * spark.shuffle.memoryFraction(默认0.2) * spark.shuffle.safetyFraction(默认0.8)
```

如果设置spark.shuffle.spill为true，则当这部分内存不能满足shuffle数据的缓存需求时，会将超出的部分存储到磁盘。

spark.shuffle/storage.safetyFraction 作用：
提供一个安全使用这部分内存的门限值，剩余部分起缓冲作用。

如何计算Spark应用申请的总内存值?

> total_memory = total_executor_memory + total_driver_memory
> = num-executors * (executor.memory + executor.memoryOverhead) + (driver.memory + driver.memoryOverhead)
> = num-executors * (executor.memory + max(10% * executor.memory, executor,memoryOverhead)) + (driver.memory + max(10% * driver.memory, driver.memoryOverhead))

最后 total_memory向上取256M的整数倍（此为简单算法，实际上是executor及driver的单内存申请量是256的向上去整数倍）。

举例：

driver.memory = 8g

executor.memory = 12g

spark.executor.memoryOverhead=4g

num-executors = 100

total_memory = [(100 * (12g + 4g) + 8g + 0.1 * 8g) / 256m] * 256m = [(1600 + 8.8) / 256 gb/mb] * 256mb = 1609 gb

内容来源：csdn.net
作者昵称：左林右李02
原文链接：https://blog.csdn.net/u011624157/article/details/123690957
作者主页：https://blog.csdn.net/u011624157